

Phase 1 - Week 2 - Lesson Day 4

GroupBy & Merging + Week 2 Mini Challenge

Today's Objectives

This is the final lesson day of Week 2. Today covers how to summarize data by category (GroupBy - exactly like a Pivot Table) and how to combine multiple tables together (merging and concatenating - exactly like SQL joins and Excel's VLOOKUP/append operations). These are the tools that turn scattered raw data into a single, analysis-ready table.

- Group data by one or more columns and summarize it
- Merge two DataFrames together based on a shared column
- Concatenate (stack) DataFrames together
- Build a pivot table

1. GroupBy

Definition: `.groupby()` splits your data into groups based on the values in one or more columns, then lets you apply a summary calculation (sum, mean, count, etc.) separately to each group. This is exactly the same concept as a Pivot Table in Excel/Power BI, or a **GROUP BY** clause in SQL.

```
df.groupby("city")["age"].mean()    # average age, grouped by city
df.groupby("city")["age"].sum()     # total age, grouped by city
df.groupby("city").size()           # count of rows per city

# Multiple aggregations at once
df.groupby("city")["age"].agg(["mean", "sum", "count"])

# Group by multiple columns
df.groupby(["city", "gender"])["age"].mean()
```

Connecting to what you already know: if you've ever dragged a field into the "Rows" area and another into "Values" in an Excel Pivot Table, you've already done exactly this - `groupby()` is that same operation written as code.

2. Merging DataFrames

Definition: `.merge()` combines two DataFrames based on a shared column (called a "key") - functionally identical to a SQL **JOIN**, or Excel's VLOOKUP/XLOOKUP but for entire tables at once instead of a single value.

```
orders = pd.DataFrame({"order_id": [1, 2, 3], "customer_id": [101, 102, 101]})
customers = pd.DataFrame({"customer_id": [101, 102], "name": ["Ali", "Halimo"]})

merged = orders.merge(customers, on="customer_id")
```

```
# combines both tables using 'customer_id' as the shared key
```

Just like SQL, there are different join types, controlled by the **how** parameter:

how=	Behavior
"inner" (default)	Keep only rows where the key exists in BOTH tables
"left"	Keep all rows from the left table, matching where possible
"right"	Keep all rows from the right table, matching where possible
"outer"	Keep everything from both tables, filling gaps with NaN

3. Concatenating DataFrames

Definition: `.concat()` stacks DataFrames together, either adding more rows (like combining January and February's sales data into one table) or more columns.

```
jan_data = pd.DataFrame({"sales": [100, 200]})
feb_data = pd.DataFrame({"sales": [150, 250]})

combined = pd.concat([jan_data, feb_data])           # stack ROWS on top of each other
combined = pd.concat([jan_data, feb_data], axis=1)   # stack side by side as new COLUMNS
```

merge() vs concat(): use **merge()** when you're combining two tables based on a matching key/ID (like joining orders to customers). Use **concat()** when you're simply stacking similar tables together (like combining 12 months of the same report into one).

4. Pivot Tables

Definition: `.pivot_table()` reshapes and summarizes data in one step - functionally identical to building a Pivot Table in Excel or Power BI.

```
df.pivot_table(
    values="sales",      # what to summarize
    index="city",        # becomes the ROWS
    columns="product",   # becomes the COLUMNS
    aggfunc="sum"        # HOW to summarize (sum, mean, count, etc.)
)
```

Today's Exercise

This exercise combines everything from all four Pandas lessons this week. Use your Day 1-3 dataset for Part A, and the small sample tables provided in the notebook for Part B (merging).

Part A - GroupBy (3 questions)

1. Group and average

Group your dataset by one categorical column and calculate the mean of a numeric column for each group.

2. Multiple aggregations

For the same grouping, calculate the mean, sum, and count all at once using `.agg()`.

3. Group by two columns

Group your dataset by two categorical columns at once and calculate the average of a numeric column for each combination.

Part B - Merging & Concatenating (3 questions)

4. Inner merge

Using the sample orders and customers tables in the notebook, merge them on the shared key and print the result.

5. Left merge

Add one extra order with a customer_id that doesn't exist in the customers table, then merge using how='left' and observe what happens to that row.

6. Concatenate two tables

Create two small DataFrames with the same columns (e.g. two months of sales), and concatenate them into one combined table.

Part C - Week 2 Mini Challenge (Sales Analysis Pipeline)

This challenge combines all four days of Week 2: loading data, filtering, cleaning, and now grouping/merging, into one connected mini pipeline.

1. Load your Day 1-3 dataset (already cleaned from Day 3).
2. Filter it down to a meaningful subset using a condition (from Day 2).
3. Group the filtered data by one categorical column and calculate at least two aggregations (mean and count).
4. Sort the grouped result from highest to lowest.
5. Write 2-3 sentences (in a markdown cell) summarizing what the grouped result tells you about the data.

How to Approach It

For the merge exercises, always double check which column is the shared key in both tables before merging - a mismatched key name (e.g. 'customer_id' vs 'CustomerID') is the most common reason a merge silently returns fewer rows than expected.

Once you and Halimo have completed all three parts, bring your answers back to the lesson chat. This wraps up Week 2 - next up is Week 3: Visualization + Descriptive Statistics.